



Module : Apprentissage Automatique

Filière : BDIA'1 — Année universitaire 2025-2026

Chapitre 2

Régression linéaire

Prof. Dr Naoual Attaoui

1. Introduction

La régression linéaire est l'un des algorithmes fondamentaux de l'apprentissage automatique supervisé. Elle vise à modéliser la relation entre une variable cible continue (appelée variable dépendante) et une ou plusieurs variables explicatives (appelées variables indépendantes ou features) au moyen d'une fonction linéaire.

Malgré sa simplicité apparente, la régression linéaire reste largement utilisée dans de nombreux domaines : prédiction de prix immobiliers, estimation de revenus, analyse de tendances financières, modélisation de phénomènes physiques, consommation électrique, etc. Elle constitue également une base essentielle pour comprendre des modèles plus avancés.

L'idée générale consiste à chercher une relation mathématique simple entre les entrées et la sortie. Dans le cas le plus simple, cette relation est une droite. En dimension plus élevée, elle devient un hyperplan. Le mot « linéaire » indique que le modèle est linéaire par rapport à ses paramètres.

La régression linéaire répond à la question suivante : étant donné un ensemble de données d'entraînement, comment trouver la meilleure droite (ou hyperplan) qui approxime la relation entre les entrées et la sortie ?

✉ **Fil conducteur — Immobilier** : Tout au long de ce chapitre, nous utiliserons la prédiction du prix d'un appartement en fonction de sa surface comme exemple fil conducteur.

2. Formulation mathématique

2.1. Régression linéaire simple

Dans le cas simple, on dispose d'une seule variable explicative x . Le modèle s'écrit :

$$\hat{y} = w_0 + w_1 \cdot x$$

où :

- w_0 : le biais (intercept, ordonnée à l'origine) ;
- w_1 : la pente, donc l'effet de x sur y (le poids);
- $\hat{y}$: la valeur prédite du modèle.

✉ **Fil conducteur — Immobilier** : $\hat{y} = w_0 + w_1 \cdot \text{surface}$. Si $w_0 = 50\,000$ et $w_1 = 2\,500$, le modèle prévoit un prix de base de 50 000 DH, puis ajoute 2 500 DH par mètre carré. Ainsi, un appartement de 60 m² coûte environ $50\,000 + 2\,500 \times 60 = 200\,000$ DH.

2.2. Régression linéaire multiple

Lorsque l'on dispose de n variables explicatives, le modèle se généralise sous forme vectorielle :

$$\hat{y} = X \cdot w$$

où

- X est la matrice des observations de taille $(m \times (n+1))$ (avec colonne de 1 pour le biais)
- w est le vecteur des poids de taille $(n+1)$
- m est le nombre d'échantillons.

Cette écriture compacte, la plus utilisée en pratique, permet :

- de représenter toutes les observations d'un coup ;
- de simplifier les calculs ;
- de préparer l'optimisation et l'implémentation numérique.

Dimensions

Si on a :

- m observations
- n variables explicatives

alors :

$$X \in \mathbb{R}^{m \times (n+1)}, \quad w \in \mathbb{R}^{(n+1) \times 1}, \quad \hat{y} \in \mathbb{R}^{m \times 1}$$

On met souvent une **colonne de 1** dans X pour intégrer le biais w_0

Forme détaillée

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1n} \\ 1 & x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix}, \quad w = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_n \end{bmatrix}$$

Donc :

$$\hat{y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_m \end{bmatrix} = Xw$$

et chaque ligne donne :

$$\hat{y}_i = w_0 + w_1 x_{i1} + w_2 x_{i2} + \cdots + w_n x_{in}$$

Exemple simple

Supposons 3 observations et 2 variables :

$$X = \begin{bmatrix} 1 & 2 & 5 \\ 1 & 3 & 4 \\ 1 & 1 & 7 \end{bmatrix}, \quad w = \begin{bmatrix} 10 \\ 2 \\ -1 \end{bmatrix}$$

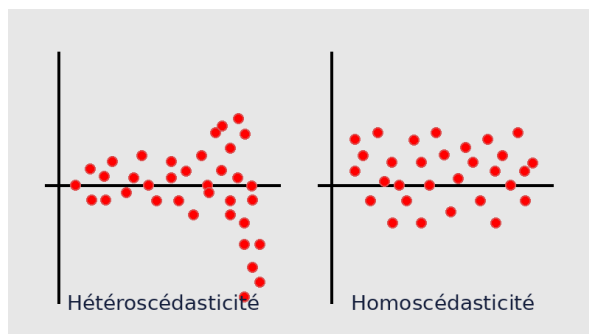
Alors :

$$\hat{y} = Xw = \begin{bmatrix} 1 & 2 & 5 \\ 1 & 3 & 4 \\ 1 & 1 & 7 \end{bmatrix} \begin{bmatrix} 10 \\ 2 \\ -1 \end{bmatrix} = \begin{bmatrix} 10 + 4 - 5 \\ 10 + 6 - 4 \\ 10 + 2 - 7 \end{bmatrix} = \begin{bmatrix} 9 \\ 12 \\ 5 \end{bmatrix}$$

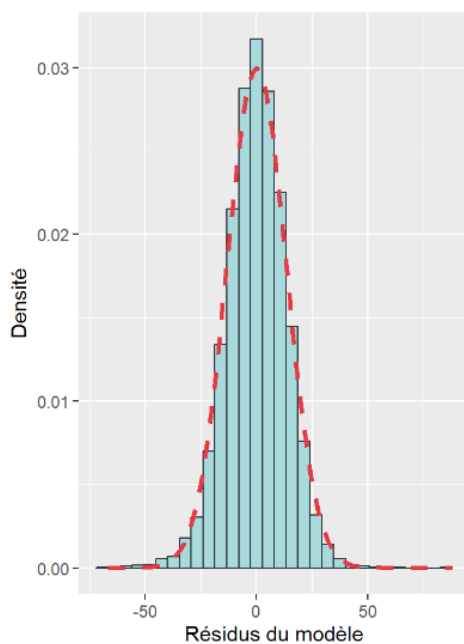
2.3. Hypothèses du modèle

Le modèle de régression linéaire repose sur plusieurs hypothèses fondamentales :

- **Linéarité** : la relation entre les variables explicatives et la cible est linéaire.
- **Indépendance** : les erreurs (résidus) sont indépendantes les unes des autres.
- **Homoscédasticité** : la variance des erreurs est constante.



- **Normalité** : les erreurs suivent une distribution normale (les petites erreurs sont les plus fréquentes).



- **Absence de multicollinéarité** : les variables explicatives ne sont pas fortement corrélées entre elles.

Ces hypothèses ne sont pas juste théoriques : elles servent à savoir quand le modèle est fiable et quand il faut être prudent dans l'interprétation.

3. Fonction coût

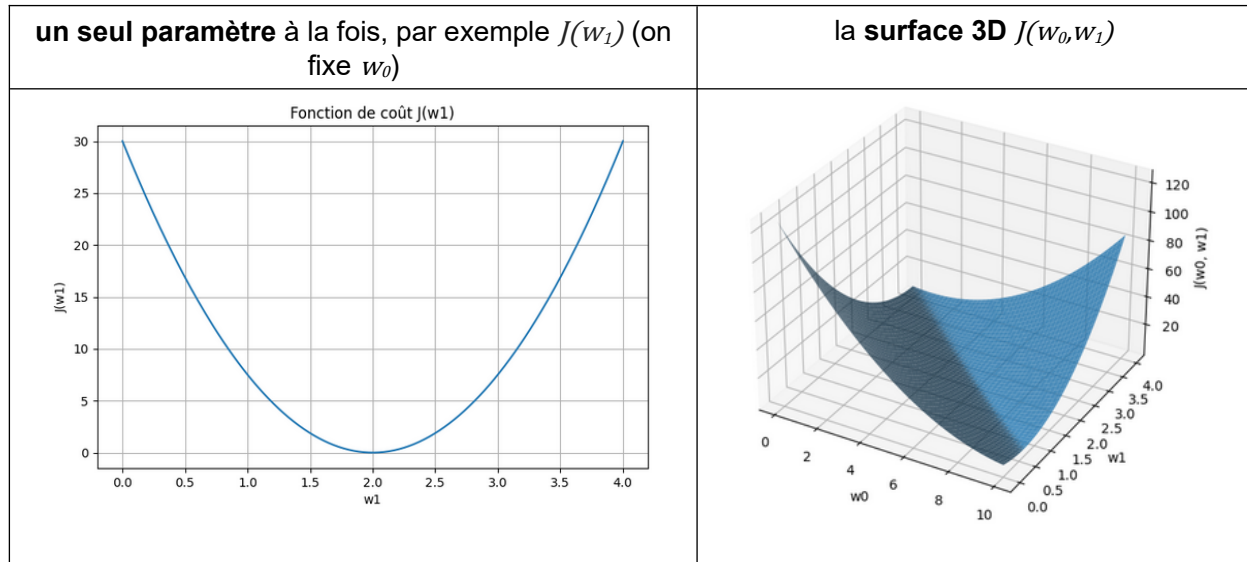
La fonction coût (ou fonction de perte) mesure l'écart entre les prédictions du modèle et les valeurs réelles.

3.1. Erreur quadratique moyenne (MSE)

Le choix le plus classique consiste à minimiser la moyenne des carrés des résidus. Cette quantité est souvent appelée erreur quadratique moyenne, ou MSE (Mean Squared Error) :

$$J(w) = (1/m) \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

$$= (1/m) \sum_{i=1}^m (x_i^T w - y_i)^2$$



L'objectif de l'apprentissage est de trouver le vecteur de poids w qui minimise cette fonction coût.

3.2. Pourquoi l'erreur quadratique ?

- Le carré élimine les signes et empêche les erreurs positives et négatives de se compenser.
- Pénalise davantage les grandes erreurs (effet du carré).
- Fonction différentiable partout, ce qui facilite l'optimisation par descente de gradient.
- Il donne une interprétation géométrique très riche dans l'espace des observations.
- Sous l'hypothèse de bruit gaussien, minimiser le MSE équivaut à maximiser la vraisemblance.

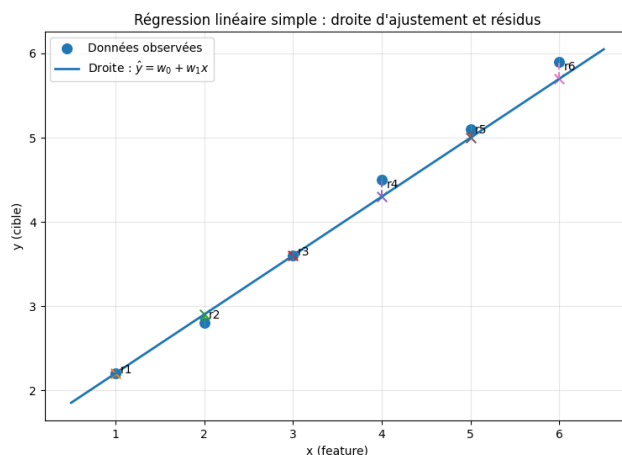
✉ **Fil conducteur — Immobilier** : Si pour 3 appartements, $\hat{y} = (180k, 250k, 310k)$ et $y = (200k, 240k, 300k)$, alors $MSE = [(20k)^2 + (10k)^2 + (10k)^2] / 3 = 200 \cdot 10^6 / 3 \approx 66,7 \cdot 10^6$.

4. Interprétation géométrique

L'interprétation géométrique donne une intuition visuelle essentielle. Géométriquement, la régression linéaire simple cherche la droite qui minimise la somme des distances verticales au carré entre chaque point de données et la droite de régression.

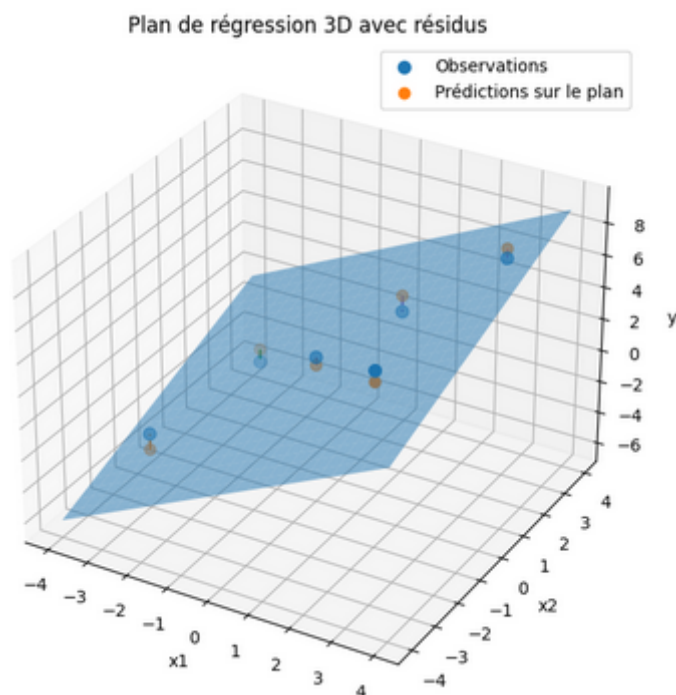
4.1. Cas simple (2D)

Dans un espace à deux dimensions (une feature, une cible), le modèle cherche la droite d'équation $y = w_0 + w_1 x$ qui passe « au plus près » de tous les points. Les résidus (erreurs) sont les segments verticaux entre chaque point et la droite. Plus ces résidus sont petits, meilleur est l'ajustement.



4.2. Cas multiple (nD)

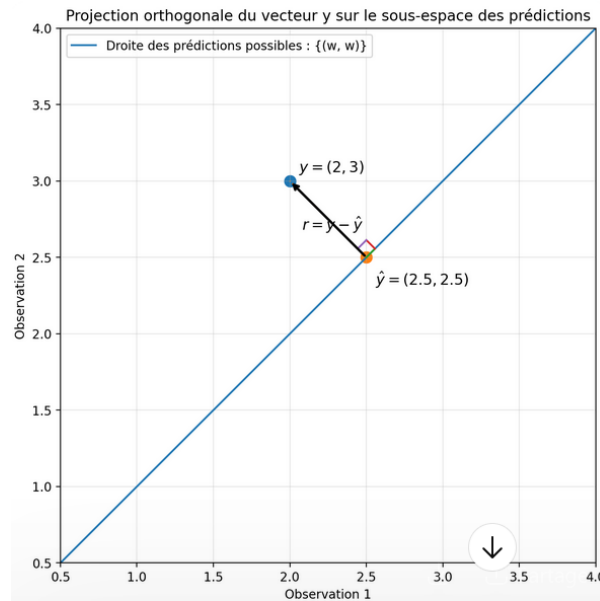
Dans un espace tridimensionnel, lorsque l'on dispose de deux variables explicatives et d'une variable cible, la régression linéaire cherche le plan d'équation $y = w_0 + w_1x_1 + w_2x_2$, qui s'ajuste au mieux au nuage de points. Les résidus correspondent aux écarts verticaux entre les points observés et ce plan. Plus ces écarts sont faibles, meilleure est la qualité de l'ajustement.



En dimension supérieure (n features), le modèle cherche un hyperplan dans un espace à $n+1$ dimensions. On ne peut plus dessiner facilement, on passe alors à une vision algébrique:

Le vecteur $\hat{y} = Xw$ est la projection orthogonale du vecteur y sur l'espace colonne de X ($Col(X)$: l'ensemble de toutes les prédictions possibles du modèle).

Le vecteur résidu $(y - \hat{y})$ est perpendiculaire à l'espace colonne de X , ce qui mène à l'équation normale $(X^T(y - \hat{y}) = 0$ donc $X^T(y - Xw) = 0$ d'où l'équation normale : $X^T Xw = X^T y$).



4.3. Synthèse géométrique

Dimension	Modèle géométrique	Minimisation
1 feature	Droite dans le plan	Distance verticale au carré
2 features	Plan dans l'espace 3D	Distance verticale au carré
n features	Hyperplan en (n+1)D	Projection orthogonale sur $\text{Col}(X)$

5. Méthodes de résolution

Résoudre un problème de régression linéaire revient à trouver les paramètres qui minimisent la fonction coût. Deux grandes familles de méthodes sont utilisées : les méthodes analytiques et les méthodes itératives.

5.1. Solution analytique : équation normale

À partir de la fonction de coût

$$\begin{aligned}
 J(w) &= (1/m) \sum_{i=1}^m (\hat{y}_i - y_i)^2 \\
 &= (1/m) \sum_{i=1}^m (x_i^T w - y_i)^2
 \end{aligned}$$

on passe à l'écriture matricielle :

$$J(w) = 1/m (Xw - y)^T (Xw - y)$$

En développant cette expression, on obtient

$$J(w) = 1/m (w^T X^T X w - 2y^T X w + y^T y)$$

Le gradient par rapport à w vaut alors

$$\nabla J(w) = (2/m) \cdot X^T (Xw - y)$$

En annulant ce gradient pour caractériser le minimum, on obtient

$$X^T(Xw - y) = 0$$

ce qui conduit à l'équation normale :

$$X^T X w = X^T y$$

Lorsque la matrice $X^T X$ est inversible, la solution optimale s'écrit

$$w^* = (X^T X)^{-1} X^T y$$

Cette solution donne directement les paramètres optimaux sans itération :

- L'inversion de matrice est coûteuse en grande dimension.
- Quand $X^T X$ est quasi-singulière (elle est inversible en théorie, mais très proche de ne plus l'être), l'ordinateur peut calculer une solution très peu fiable, car le problème devient trop sensible aux petites erreurs numériques.
- Si vous avez plus de variables que d'exemples, il n'y a pas assez d'information pour déterminer une solution unique par cette formule classique.

5.2. Descente de gradient

La descente de gradient est une méthode itérative qui ajuste progressivement les paramètres dans la direction opposée au gradient de la fonction coût :

$$w := w - \alpha \cdot \nabla J(w)$$

où α est le taux d'apprentissage (learning rate).

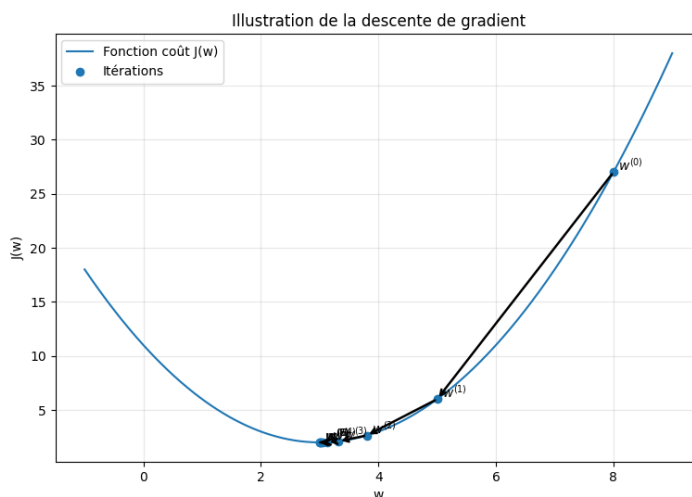
Le gradient pour le MSE est : $\nabla J(w) = (2/m) \cdot X^T(Xw - y)$

Le vecteur $Xw - y$ contient les erreurs faites par le modèle sur toutes les observations. En multipliant ce vecteur par X^T , on mesure l'effet de ces erreurs sur chacun des coefficients. Le gradient joue donc le rôle d'un indicateur de correction : il dit, pour chaque paramètre, s'il faut l'augmenter, le diminuer, et avec quelle intensité.

La descente de gradient consiste alors à répéter cette correction pas à pas, jusqu'à atteindre un minimum.

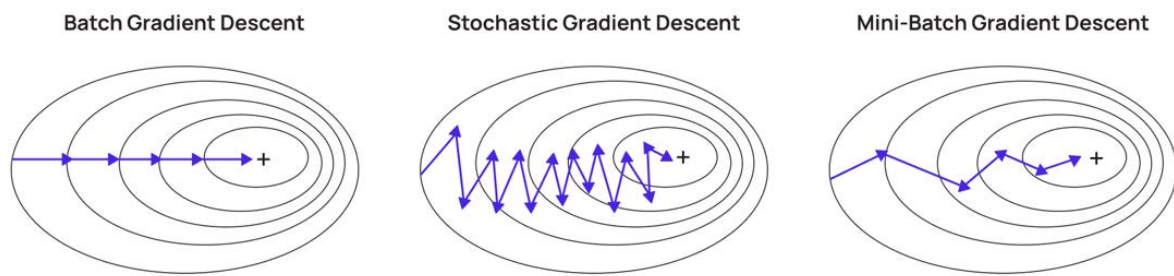
L'idée est donc la suivante : à chaque itération, on calcule les erreurs, on en déduit le gradient, puis on met à jour les coefficients dans la direction qui fait diminuer l'erreur.

Cette approche est très générale et devient particulièrement utile lorsque la taille des données rend la solution analytique coûteuse.



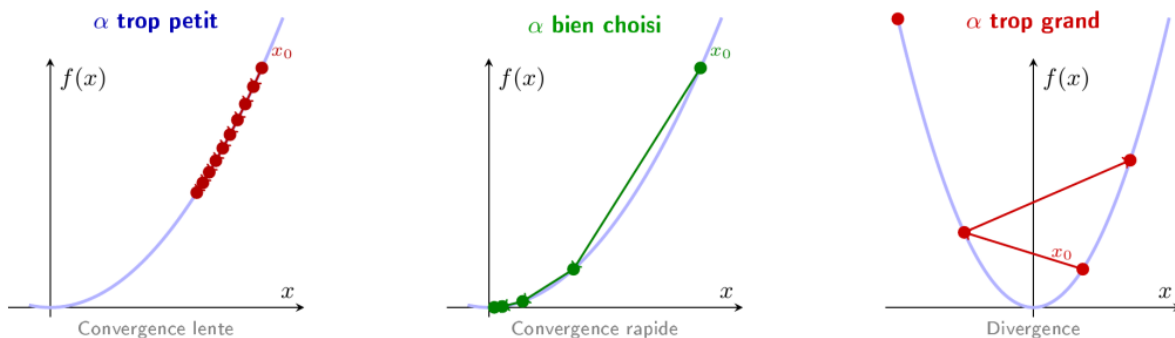
5.2.1. Variantes de la descente de gradient

Variante	Données par itération	Avantages	Inconvénients
Batch GD	Tout le dataset	Convergence stable	Lent pour grands datasets
Stochastic GD (SGD)	1 échantillon	Rapide, évite minima locaux	Convergence bruyante
Mini-batch GD	Lot de taille B	Compromis vitesse/stabilité	Hyperparamètre B



5.2.2 Choix du taux d'apprentissage

- α trop petit : convergence très lente.
- α trop grand : oscillations, divergence possible.



5.3. Comparaison des méthodes

Critère	Équation normale	Descente de gradient
Type	Solution fermée	Itératif
Grande dimension	Coûteuse	Efficace
Hyperparamètres	Aucun	α , nombre d'itérations
Passage à l'échelle	Limité	Très bon (SGD)

📧 **Fil conducteur — Immobilier** : Pour un petit dataset (100 appartements, 3 features), l'équation normale suffit. Pour 1 million d'annonces avec 50 features, la descente de gradient est préférable.

6. Types de régression linéaire

6.1. Régression linéaire simple

Une seule variable explicative. Le modèle est une droite : $\hat{y} = w_0 + w_1x$. C'est le cas le plus intuitif et le plus facile à visualiser. Elle est particulièrement utile pour les jeux de données où une relation dominante est observée.

6.2. Régression linéaire multiple

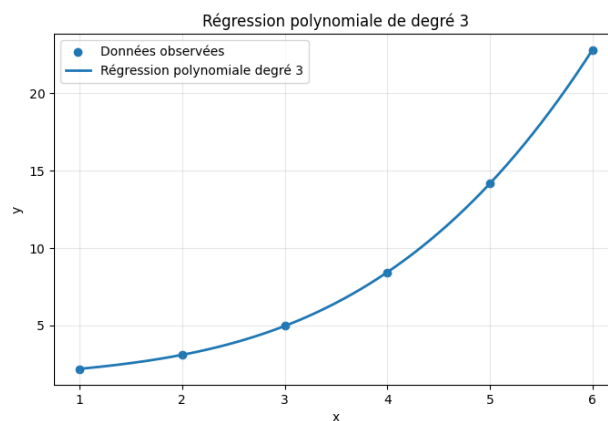
Plusieurs variables explicatives : $\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$. Chaque coefficient w_i représente l'effet marginal de la variable x_i sur la cible, toutes les autres variables étant égales par ailleurs. C'est le cas le plus fréquent en pratique, car un phénomène réel dépend rarement d'un seul facteur.

6.3. Régression polynomiale

Bien que le mot « polynomiale » puisse suggérer un modèle non linéaire, ce modèle reste linéaire par rapport aux paramètres :

$$\hat{y} = w_0 + w_1x + w_2x^2 + w_3x^3 + \dots$$

On crée de nouvelles features (x^2 , x^3 , ...) puis on applique la régression linéaire standard.



6.4. Synthèse

Type	Nombre de features	Relation	Complexité du modèle
Simple	1	Linéaire	Faible
Multiple	$n \geq 2$	Linéaire	Moyenne
Polynomiale	1 (transformée)	Non linéaire en x	Élevée (risque overfitting)

Remarque : Plus le modèle devient flexible, plus le risque de surapprentissage augmente.

7. Métriques d'évaluation

Pour évaluer la qualité d'un modèle de régression, on utilise plusieurs métriques complémentaires :

7.1. Erreur quadratique moyenne (MSE)

- **Formule :**

$$\text{MSE} = (1/m) \sum (y - \hat{y})^2$$

- **Interprétation:**

La MSE mesure l'erreur moyenne entre les valeurs réelles et les valeurs prédites, en donnant davantage de poids aux grandes erreurs à cause du carré.

- **Avantages :**

- Très utilisée en apprentissage automatique.
- Dérivable, donc pratique pour l'optimisation mathématique.
- Pénalise fortement les erreurs importantes.

- **Limites :**

- Son unité est au carré, ce qui la rend moins intuitive à interpréter.
- Elle est sensible aux valeurs aberrantes.

- **Remarques :**

- L'unité de la MSE est le carré de l'unité de la variable cible.
- En régression linéaire, c'est souvent cette métrique qui est minimisée pendant l'apprentissage.

7.2. Racine de l'erreur quadratique moyenne (RMSE)

- **Formule :**

$$\text{RMSE} = \sqrt{\text{MSE}}$$

- **Interprétation :**

La RMSE exprime l'erreur moyenne dans la même unité que la variable cible, ce qui la rend plus facile à interpréter que la MSE.

- **Avantages :**

- Plus interprétable que la MSE.
- Exprimée dans la même unité que la cible.
- Continue à pénaliser fortement les grandes erreurs.

- **Limites :**

- Elle reste sensible aux valeurs aberrantes.
- Une erreur très grande peut fortement influencer sa valeur.

- **Remarques :**

- La RMSE est souvent préférée à la MSE lorsque l'on souhaite une métrique plus lisible.

7.3. Erreur absolue moyenne (MAE)

- **Formule :**

$$\text{MAE} = (1/m) \sum |y - \hat{y}|$$

- **Interprétation :**
La MAE indique de combien, en moyenne, les prédictions s'écartent des vraies valeurs, en valeur absolue.
- **Avantages :**
 - Facile à comprendre.
 - Exprimée dans la même unité que la variable cible.
 - Moins sensible aux erreurs extrêmes que la MSE ou la RMSE.
- **Limites :**
 - Elle ne pénalise pas fortement les grosses erreurs.
 - Elle distingue moins sévèrement les erreurs modérées et les erreurs très importantes.
- **Remarques :**
 - La MAE est plus robuste aux valeurs aberrantes que la MSE.
 - Exemple : si la MAE vaut 2,5, cela signifie que le modèle se trompe en moyenne de 2,5 unités.

7.4. Coefficient de détermination (R^2)

- **Formule :**

$$R^2 = 1 - \frac{\sum_{i=1}^m (y_i - \hat{y}_i)^2}{\sum_{i=1}^m (y_i - \bar{y})^2}$$

- **Interprétation :**
Le coefficient R^2 mesure la proportion de la variance de la variable cible expliquée par le modèle.
- **Avantages :**
 - Très utilisé pour juger la qualité globale de l'ajustement.
 - Donne une idée du pouvoir explicatif du modèle.
- **Limites :**
 - Un R^2 élevé ne garantit pas que le modèle soit bon dans toutes les situations.
 - Il peut être artificiellement amélioré lorsqu'on ajoute trop de variables.
- **Remarques :**
 - $R^2=1$: modèle parfait.
 - $R^2=0$: le modèle n'explique rien de plus que la moyenne.
 - $R^2<0$: le modèle est pire qu'une prédiction par la moyenne.

7.5. R^2 ajusté

- **Formule :**

$$R^2_{adj} = 1 - [(1-R^2)(m-1)] / (m-n-1)$$

- **Interprétation:**
Le R^2 ajusté corrige le R^2 classique en tenant compte du nombre de variables explicatives n et du nombre d'échantillons m .
- **Avantages :**
 - Pénalise l'ajout de variables inutiles.
 - Très utile en régression linéaire multiple pour comparer plusieurs modèles.
- **Limites :**

- Ne suffit pas, à lui seul, à juger complètement la qualité d'un modèle.
- Peut rester insuffisant si les hypothèses du modèle sont mal respectées.
- **Remarques :**
 - Le R^2 ajusté est souvent plus pertinent que le R^2 simple lorsque plusieurs variables explicatives sont utilisées.
 - Il évite de surestimer artificiellement la qualité d'un modèle trop chargé en variables.

7.6. Quelle métrique choisir ?

Le choix dépend de l'objectif.

On préfère souvent :

- **MAE** si l'on veut une mesure simple et robuste
- **RMSE** si les grosses erreurs sont particulièrement graves
- **R2** pour mesurer le pouvoir explicatif global
- **R2 ajusté** en présence de plusieurs variables

7.7. Lecture critique

- **MSE** est utile pour l'optimisation, mais moins intuitive.
- Une faible MAE signifie que l'erreur moyenne absolue est faible.
- Une faible RMSE indique également de bonnes prédictions, mais avec une sensibilité plus forte aux grandes erreurs.
- Un R^2 proche de 1 indique que le modèle explique bien la variabilité de la cible.
- Aucune métrique ne suffit seule : il faut toujours les interpréter dans le contexte métier ou physique.

✉ **Fil conducteur — Immobilier** : Un $R^2 = 0,85$ signifie que le modèle explique 85 % de la variance des prix. Un RMSE = 15 000 DH indique une erreur moyenne de 15 000 DH sur les prédictions.

8. Régularisation

Lorsque le modèle devient trop flexible ou lorsque les variables explicatives sont nombreuses et corrélées, il peut surapprendre les données d'entraînement. La régularisation ajoute alors une pénalité sur les coefficients afin de limiter leur amplitude.

8.1. Régression Ridge (L2)

La régularisation Ridge ajoute une pénalité proportionnelle à la somme des carrés des coefficients $\lambda \sum w_j^2$ à la fonction coût :

$$J_{\text{Ridge}}(w) = (1/m) \sum (y_i - \hat{y}_i)^2 + \lambda \sum w_j^2$$

- Réduit les poids vers zéro sans les annuler complètement.
- Ridge est particulièrement utile quand les variables explicatives sont nombreuses ou fortement corrélées.
- Améliore la stabilité numérique (résout le problème de multicollinéarité).
- Solution analytique : $w^* = (X^T X + \lambda I)^{-1} X^T y$.

8.2. Régression Lasso (L1)

La régularisation Lasso ajoute une pénalité proportionnelle à la somme des valeurs absolues des coefficients $\lambda \sum |w_j|$:

$$J_{\text{Lasso}}(w) = (1/m) \sum (y_i - \hat{y}_i)^2 + \lambda \sum |w_j|$$

- Peut annuler complètement certains poids (sélection automatique de variables).
- Lasso est utile quand on pense que seules quelques variables sont réellement importantes.
- Produit des modèles parcimonieux (sparse).
- Pas de solution analytique, résolution par algorithmes itératifs.

8.3. Elastic Net

L'Elastic Net combine les pénalités L1 et L2 :

$$J_{\text{EN}}(w) = (1/m) \sum (y_i - \hat{y}_i)^2 + \lambda_1 \sum |w_j| + \lambda_2 \sum w_j^2$$

L'Elastic Net hérite des avantages des deux méthodes : sélection de variables (L1) et stabilité (L2).

8.4. Effet du paramètre λ

- $\lambda = 0$: pas de régularisation (régression linéaire standard).
- λ petit : faible régularisation, risque de surapprentissage.
- λ grand : forte régularisation (les coefficients sont fortement réduits), risque de sous-apprentissage.
- Choix optimal : par validation croisée (cross-validation).

8.5. Comparaison

Méthode	Pénalité	Sélection de variables	Solution analytique
Ridge (L2)	$\lambda \sum w_j^2$	Non	Oui
Lasso (L1)	$\lambda \sum w_j $	Oui (peut annuler certains coefficients)	Non
Elastic Net	$\lambda_1 w + \lambda_2 w^2$	Oui	Non

8.6. Effet sur le compromis biais-variance

La régularisation :

- augmente un peu le **biais** ;
- réduit souvent la **variance** ;
- améliore donc la performance sur de nouvelles données.

C'est pour cela qu'elle est très utile en pratique.

✉ **Fil conducteur — Immobilier** : Avec 50 features (surface, étage, proximité métro, année, ...), le Lasso peut identifier les 10 features les plus importantes en mettant les autres poids à zéro.

9. Implémentation Python

9.1. Régression linéaire simple avec scikit-learn

Nous présentons ici une implémentation complète de la régression linéaire simple en Python à l'aide de scikit-learn et NumPy.

```
# Importation des bibliothèques
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# 1) Données : surface (m²) et prix (DH)
# reshape(-1,1) : transforme en matrice colonne (obligatoire pour sklearn)
X = np.array([30,40,50,60,70,80,90,100,110,120]).reshape(-1,1)
y = np.array([125000,160000,185000,210000,248000,275000,
              310000,340000,365000,400000])

# 2) Division en apprentissage / test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2
)

# 3) Entraînement du modèle
model = LinearRegression()
model.fit(X_train, y_train)

# 4) Paramètres appris
print(f"w0 (biais) = {model.intercept_:.0f}")
print(f"w1 (pente) = {model.coef_[0]:.0f}")

# 5) Prédictions
y_pred_test = model.predict(X_test)
y_pred_all = model.predict(X)

# 6) Évaluation
mse = mean_squared_error(y_test, y_pred_test)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred_test)

print(f"MSE = {mse:.0f}")
print(f"RMSE = {rmse:.0f}")
print(f"R² = {r2:.4f}")

# 7) Préparation d'une droite lisse
X_line = np.linspace(X.min(), X.max(), 200).reshape(-1, 1)
y_line = model.predict(X_line)

# 8) Visualisation améliorée
plt.figure(figsize=(8, 5))

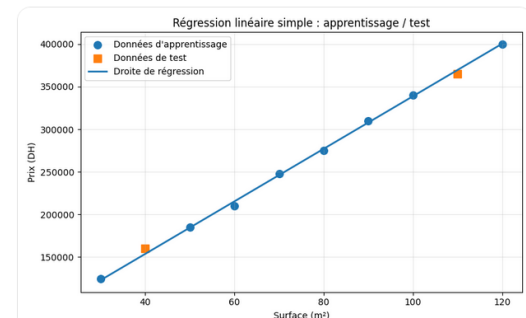
# points d'apprentissage
plt.scatter(X_train, y_train, label="Données d'apprentissage", s=70)

# points de test
plt.scatter(X_test, y_test, label="Données de test", s=70, marker="s")

# droite de régression
plt.plot(X_line, y_line, label="Droite de régression", linewidth=2)

plt.xlabel("Surface (m²)")
plt.ylabel("Prix (DH)")
plt.title("Régression linéaire simple : apprentissage / test")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
w0 (biais) = 30353
w1 (pente) = 3084
MSE = 30214960
RMSE = 5497
R² = 0.9971
```



Modèle appris : $\hat{y} = 30353 + 3084x$

Interprétation de la pente : chaque mètre carré supplémentaire augmente le prix estimé d'environ **3084 DH**.

RMSE = 5497 DH : l'erreur de prédiction reste relativement faible.

R²=0.9971 : le modèle explique très bien les données.

Graphiquement : les points sont proches de la droite, ce qui confirme une relation linéaire croissante entre la surface et le prix.

9.2. Régression multiple

Ce code illustre la régression linéaire avec plusieurs variables explicatives. Le modèle apprend simultanément l'effet de chaque variable sur le prix.

```
...
import pandas as pd # Manipulation de données sous forme de tableau

# =====
# 1. Création des données
# =====
# On crée un petit dataset représentant des logements
# Chaque ligne = un logement
# Chaque colonne = une caractéristique

data = {
    "surface": [50, 60, 70, 80, 90, 100, 110, 120], # surface en m²
    "chambres": [1, 2, 2, 3, 3, 3, 4, 4], # nombre de chambres
    "etage": [1, 2, 3, 1, 4, 5, 2, 6], # étage
    "prix": [180, 220, 250, 280, 310, 340, 370, 410] # prix (en milliers DH par ex.)
}

# Conversion en DataFrame (format tableau)
df = pd.DataFrame(data)

# =====
# 2. Séparation des variables
# =====
# X = variables explicatives (features)
# y = variable cible (target)

X = df[["surface", "chambres", "etage"]] # entrées du modèle
y = df["prix"] # sortie à prédire

# =====
# 3. Création et apprentissage du modèle
# =====
# On crée un modèle de régression linéaire
model = LinearRegression()

# Entraînement du modèle :
# Le modèle apprend les coefficients de l'équation
model.fit(X, y)

# =====
# 4. Prédiction
# =====
# Le modèle prédit les prix à partir des données d'entrée
y_pred = model.predict(X)

# =====
# 5. Affichage des paramètres du modèle
# =====
print("=== Régression linéaire multiple ===")

# w0 : constante (biais)
print(f"Intercept (w0) = {model.intercept_:.4f}")

# w1, w2, w3 : coefficients associés à chaque variable
for feature, coef in zip(X.columns, model.coef_):
    print(f"Coefficient de {feature} = {coef:.4f}")

# Le modèle appris est de la forme :
# prix = w0 + w1*surface + w2*chambres + w3*etage

# =====
# 6. Évaluation du modèle
# =====
mse = mean_squared_error(y, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y, y_pred)
r2 = r2_score(y, y_pred)
```

```
=== Régression linéaire multiple ===
Intercept (w0) = 37.6744
Coefficient de surface = 2.5640
Coefficient de chambres = 11.2791
Coefficient de etage = 2.7907
MSE = 3.4157
RMSE = 1.8482
MAE = 1.4244
R² = 0.9994

Comparaison réel / prédit :
      surface  chambres  etage  prix  prix_predit
0         50         1      1    180    179.941860
1         60         2      2    220    219.651163
2         70         2      3    250    248.081395
3         80         3      1    280    279.418605
4         90         3      4    310    313.430233
5        100         3      5    340    341.860465
6        110         4      2    370    370.406977
7        120         4      6    410    407.209302
```

Ce code apprend une formule mathématique qui relie le prix d'un logement à sa surface, son nombre de chambres et son étage, puis vérifie à quel point cette formule reproduit les prix observés.

Point très important

En régression multiple, un coefficient ne s'interprète jamais seul de manière naïve. Il s'interprète toujours **“toutes choses égales par ailleurs”**. Par exemple, si le coefficient de surface vaut 2.3, cela signifie :

à chambres et étage fixés, 1 unité de surface supplémentaire augmente le prix prédit de 2.3.


```
# Affichage des métriques
print(f"MSE = {mse:.4f}")
print(f"RMSE = {rmse:.4f}")
print(f"MAE = {mae:.4f}")
print(f"R² = {r2:.4f}")

# =====
# 7. Comparaison réel vs prédit
# =====
# On ajoute les prédictions au tableau initial
results = df.copy()
results["prix_predit"] = y_pred

print("\nComparaison réel / prédit :")
print(results)
```

9.3. Régression polynomiale

Ce code illustre la régression polynomiale avec une variable explicative transformée. Le modèle apprend une relation non linéaire entre l'entrée et la sortie en ajoutant des puissances de x , ici le terme x^2 .

```
..
from sklearn.preprocessing import PolynomialFeatures # Transformation
polynomiale

# =====
# 1. Données
# =====
# Variable explicative (entrée)
X = np.array([1, 2, 3, 4, 5, 6]).reshape(-1, 1)

# Variable cible (sortie)
y = np.array([1.5, 3.8, 8.5, 15.5, 24.0, 35.5])

# Observation :
# Les valeurs de y augmentent de plus en plus vite → relation non linéaire

# =====
# 2. Transformation polynomiale
# =====

# On crée des nouvelles variables : x et x²
# include_bias=False Parce que la régression linéaire ajoute déjà elle-même
# l'intercept w0.
poly = PolynomialFeatures(degree=2, include_bias=False)

# Transformation des données :
# Exemple : x=3 → [3, 9]
X_poly = poly.fit_transform(X)

# Maintenant le modèle travaille avec :
# X_poly = [x, x²]

# =====
# 3. Création et apprentissage du modèle
# =====

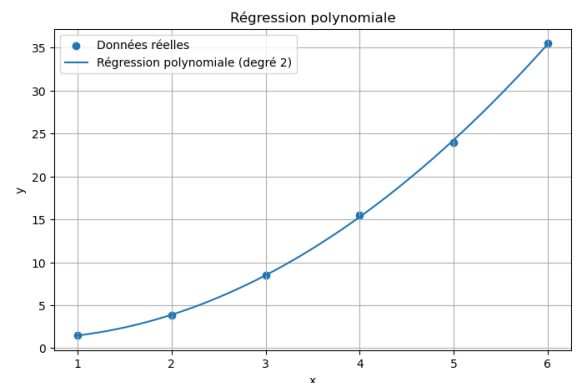
# On utilise une régression linéaire classique
model = LinearRegression()

# Entraînement :
# Le modèle apprend les coefficients de :
# y = w0 + w1*x + w2*x²
model.fit(X_poly, y)

# =====
# 4. Prédictions
# =====

# Calcul des valeurs prédites pour les données existantes
y_pred = model.predict(X_poly)
```

```
=== Régression polynomiale ===
Intercept (w0) = 1.2400
Coefficients = [-0.86142857  1.09285714]
MSE = 0.0215
RMSE = 0.1467
MAE = 0.1143
R² = 0.9998
```



Dans cet exemple, les points ne suivent pas une droite mais une courbe. On ajoute donc une nouvelle variable x^2 . Le modèle apprend alors une parabole qui s'ajuste mieux aux données que la droite classique.

```

# =====
# 5. Affichage du modèle
# =====

print("=== Régression polynomiale ===")

# w0 : constante
print(f"Intercept (w0) = {model.intercept_:.4f}")

# w1 et w2 : coefficients
print(f"Coefficients = {model.coef_}")

# Le modèle appris est :
#  $y = w_0 + w_1x + w_2x^2$ 

# =====
# 6. Évaluation du modèle
# =====

mse = mean_squared_error(y, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y, y_pred)
r2 = r2_score(y, y_pred)

print(f"MSE = {mse:.4f}")
print(f"RMSE = {rmse:.4f}")
print(f"MAE = {mae:.4f}")
print(f"R² = {r2:.4f}")

# =====
# 7. Création d'une courbe lisse
# =====

# On crée 200 points entre min(X) et max(X)
X_new = np.linspace(X.min(), X.max(), 200).reshape(-1, 1)

# On applique la même transformation polynomiale
X_new_poly = poly.transform(X_new)

# Prédiction pour tracer une courbe fluide
y_new = model.predict(X_new_poly)

# =====
# 8. Visualisation
# =====

plt.figure(figsize=(8, 5))

# Points réels
plt.scatter(X, y, label="Données réelles")

# Courbe du modèle
plt.plot(X_new, y_new, label="Régression polynomiale (degré 2)")

plt.xlabel("x")
plt.ylabel("y")
plt.title("Régression polynomiale")
plt.grid(True)
plt.legend()
plt.show()

```

9.4. Descente de gradient

La descente de gradient implémente manuellement l'optimisation itérative des poids. Ce code montre chaque étape du processus : initialisation, calcul du gradient, mise à jour des poids, et convergence.

Important : la normalisation des features (feature scaling) est essentielle pour que la descente de gradient converge rapidement. Sans normalisation, un taux d'apprentissage adapté à une variable peut être trop grand ou trop petit pour une autre.

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# =====
# 1) Données : surface (m²) -> prix (DH)
# =====
X_raw = np.array([30, 40, 50, 60, 70, 80, 90, 100, 110, 120], dtype=float)
y = np.array([125000, 160000, 185000, 210000, 248000, 275000,
              310000, 340000, 365000, 400000], dtype=float)

# =====
# 2) Normalisation de la variable d'entrée
#     x_norm = (x - moyenne) / écart-type
# =====
X_mean = X_raw.mean()
X_std = X_raw.std()
X_norm = (X_raw - X_mean) / X_std

# =====
# 3) Construction de la matrice X
#     Première colonne : 1 (biais)
#     Deuxième colonne : x normalisé
# =====
m = len(X_raw)
X = np.column_stack((np.ones(m), X_norm)) # matrice de taille (m, 2)

# =====
# 4) Initialisation des paramètres
#     Modèle : y_hat = w0 + w1 * x_norm
# =====
w = np.zeros(2) # [w0, w1]

# =====
# 5) Hyperparamètres
# =====
alpha = 0.1 # taux d'apprentissage
n_iter = 300 # nombre d'itérations

# =====
# 6) Descente de gradient batch
#     Fonction de coût :
#     J(w) = (1/m) * somme( (y_hat - y)^2 )
#     Gradient :
#     ∇J(w) = (2/m) * X^T (Xw - y)
# =====
historique_cout = []

for i in range(n_iter):
    y_pred = X @ w # prédictions, @ = produit matriciel
    erreur = y_pred - y # résidus
    gradient = (2 / m) * (X.T @ erreur)
    w = w - alpha * gradient # mise à jour des paramètres

    cout = np.mean(erreur ** 2) # MSE
    historique_cout.append(cout)

    if i % 50 == 0:
        print(f"Itération {i:3d} | MSE = {cout:,.0f} | w0 = {w[0]:,.0f} | w1 = {w[1]:,.0f}")

# =====
# 7) Paramètres finaux sur variable normalisée
# =====
print("\n--- Modèle appris sur X normalisé ---")
print(f"w0 = {w[0]:,.4f}")
print(f"w1 = {w[1]:,.4f}")
print(f"Equation : y_hat = w0 + w1 * x_normalise")

# =====
# 8) Conversion vers les coefficients sur X brut
#     x_norm = (x - X_mean) / X_std
#     donc :
#     y_hat = w0 + w1 * (x - X_mean) / X_std

```

```

Itération 0 | MSE = 76,160,400,000 | w0 = 52,360 | w1 = 17,450
Itération 50 | MSE = 8,995,167 | w0 = 261,797 | w1 = 87,247
Itération 100 | MSE = 8,995,152 | w0 = 261,800 | w1 = 87,248
Itération 150 | MSE = 8,995,152 | w0 = 261,800 | w1 = 87,248
Itération 200 | MSE = 8,995,152 | w0 = 261,800 | w1 = 87,248
Itération 250 | MSE = 8,995,152 | w0 = 261,800 | w1 = 87,248

```

```

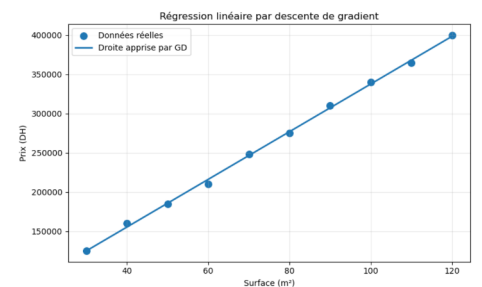
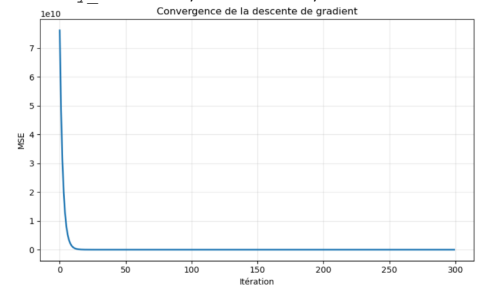
--- Modèle appris sur X normalisé ---
w0 = 261,800.0000
w1 = 87,247.7212
Equation : y_hat = w0 + w1 * x_normalise

```

```

--- Modèle équivalent sur X brut ---
w0_brut = 33,981.8182
w1_brut = 3,037.5758
Equation : y_hat = 33,981.82 + 3,037.58 * x

```



```

--- Comparaison avec scikit-learn ---
scikit-learn : w0 = 33,981.8182 | w1 = 3,037.5758
Gradient Descent converti : w0 = 33,981.8182 | w1 = 3,037.5758

```

Convergence : la courbe de la MSE décroît rapidement au début puis se stabilise progressivement, ce qui indique que l'algorithme converge vers un minimum de la fonction de coût.

Normalisation : la normalisation de la variable d'entrée améliore la stabilité numérique et accélère la convergence. Sans normalisation, la descente de gradient peut devenir plus lente, osciller, voire diverger pour certaines valeurs du taux d'apprentissage.

Taux d'apprentissage (α) : ici, $\alpha=0.1$ donne une convergence rapide et stable. Une valeur plus faible, comme $\alpha=0.01$, conduit à une convergence plus lente. Une valeur trop

```

#           = (w0 - w1*X_mean/X_std) + (w1/X_std)*x
# =====
w1_raw = w[1] / X_std
w0_raw = w[0] - (w[1] * X_mean / X_std)

print("\n--- Modèle équivalent sur X brut ---")
print(f"w0_brut = {w0_raw:,.4f}")
print(f"w1_brut = {w1_raw:,.4f}")
print(f"Equation : y_hat = {w0_raw:,.2f} + {w1_raw:,.2f} * x")

# =====
# 9) Prédiction finale sur X brut
# =====
X_line = np.linspace(X_raw.min(), X_raw.max(), 200)
y_line = w0_raw + w1_raw * X_line

# =====
# 10) Figure 1 : convergence du coût
# =====
plt.figure(figsize=(8, 5))
plt.plot(historique_cout, linewidth=2)
plt.xlabel("Itération")
plt.ylabel("MSE")
plt.title("Convergence de la descente de gradient")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# =====
# 11) Figure 2 : droite apprise
# =====
plt.figure(figsize=(8, 5))
plt.scatter(X_raw, y, label="Données réelles", s=70)
plt.plot(X_line, y_line, label="Droite apprise par GD", linewidth=2)
plt.xlabel("Surface (m²)")
plt.ylabel("Prix (DH)")
plt.title("Régression linéaire par descente de gradient")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# =====
# 12) Comparaison avec scikit-learn
# =====
model_sk = LinearRegression()
model_sk.fit(X_raw.reshape(-1, 1), y)

print("\n--- Comparaison avec scikit-learn ---")
print(f"scikit-learn : w0 = {model_sk.intercept_:,.4f} | w1 = {model_sk.coef_[0]:,.4f}")
print(f"Gradient Descent converti : w0 = {w0_raw:,.4f} | w1 = {w1_raw:,.4f}")

```

grande, comme $\alpha=1.0$, peut provoquer des oscillations ou une divergence.

Résultat : après 300 itérations, les coefficients obtenus par descente de gradient sont très proches de ceux calculés par scikit-learn, ce qui montre que l'algorithme retrouve correctement la solution de la régression linéaire.

📧 **Fil conducteur — Immobilier :** Pour un petit jeu de données immobilier (10 observations), la solution analytique (scikit-learn) est instantanée. La descente de gradient devient indispensable pour des millions d'annonces.

Descente de gradient avec *SGDRegressor*

Si on veut faire la même idée, mais sans reprogrammer manuellement la descente de gradient, le plus adapté est d'utiliser *SGDRegressor* de scikit-learn.

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import SGDRegressor, LinearRegression

# =====
# 1) Données : surface (m²) -> prix (DH)
# =====
X_raw = np.array([30, 40, 50, 60, 70, 80, 90, 100, 110, 120], dtype=float)

```

```

y = np.array([125000, 160000, 185000, 210000, 248000, 275000,
              310000, 340000, 365000, 400000], dtype=float)

# =====
# 2) Normalisation de la variable d'entrée
# =====
X_mean = X_raw.mean()
X_std = X_raw.std()
X_norm = ((X_raw - X_mean) / X_std).reshape(-1, 1)

# =====
# 3) Modèle de descente de gradient sklearn
#   attention :
#   - eta0 = taux d'apprentissage
#   - alpha correspond à la force de régularisation dans SGDRegressor
#   - ici, comme penalty=None, aucune régularisation n'est appliquée
# =====
n_iter = 300
model_gd = SGDRegressor(
    loss="squared_error", # Le modèle utilise l'erreur quadratique, comme en régression linéaire classique.

    penalty=None, # Aucune régularisation n'est ajoutée.
    learning_rate="constant", # Le taux d'apprentissage reste constant.
    eta0=0.1, # C'est le taux d'apprentissage.
    max_iter=1, # À chaque appel de fit, le modèle ne fait qu'une seule époque.
    tol=None, # On désactive le critère d'arrêt automatique.
    warm_start=True, # Le modèle garde les coefficients appris d'un appel à fit au suivant.
    random_state=42
)

# =====
# 4) Entraînement progressif pour suivre la MSE
# =====
historique_cout = []

# On lance une boucle de 300 itérations.
# À chaque tour, le modèle va apprendre un peu plus.

for i in range(n_iter):
    model_gd.fit(X_norm, y) # une époque à chaque fois grâce à max_iter=1 + warm_start=True

    y_pred = model_gd.predict(X_norm)
    mse = np.mean((y - y_pred) ** 2)
    historique_cout.append(mse)

    if i % 50 == 0: # Affichage partiel toutes les 50 itérations
        print(
            f"Itération {i:3d} | MSE = {mse:,.0f} | "
            f"w0 = {model_gd.intercept_[0]:,.0f} | w1 = {model_gd.coef_[0]:,.0f}"
        )

# =====
# 5) Paramètres finaux sur variable normalisée
# =====
w0 = model_gd.intercept_[0]
w1 = model_gd.coef_[0]

print("\n--- Modèle appris sur X normalisé ---")
print(f"w0 = {w0:,.4f}")
print(f"w1 = {w1:,.4f}")
print("Equation : y_hat = w0 + w1 * x_normalise")

# =====
# 6) Conversion vers les coefficients sur X brut
# =====
w1_raw = w1 / X_std
w0_raw = w0 - (w1 * X_mean / X_std)

print("\n--- Modèle équivalent sur X brut ---")
print(f"w0_brut = {w0_raw:,.4f}")
print(f"w1_brut = {w1_raw:,.4f}")
print(f"Equation : y_hat = {w0_raw:,.2f} + {w1_raw:,.2f} * x")

# =====

```

```

# 7) Prédiction pour la droite finale
# =====
X_line = np.linspace(X_raw.min(), X_raw.max(), 200)
y_line = w0_raw + w1_raw * X_line

# =====
# 8) Figure 1 : convergence du coût
# =====
plt.figure(figsize=(8, 5))
plt.plot(historique_cout, linewidth=2)
plt.xlabel("Itération")
plt.ylabel("MSE")
plt.title("Convergence de la descente de gradient (SGDRegressor)")
plt.grid(True, alpha=0.1)
plt.tight_layout()
plt.show()

# =====
# 9) Figure 2 : droite apprise
# =====
plt.figure(figsize=(8, 5))
plt.scatter(X_raw, y, label="Données réelles", s=70)
plt.plot(X_line, y_line, label="Droite apprise par GD sklearn", linewidth=2)
plt.xlabel("Surface (m²)")
plt.ylabel("Prix (DH)")
plt.title("Régression linéaire par descente de gradient")
plt.legend()
plt.grid(True, alpha=0.1)
plt.tight_layout()
plt.show()

# =====
# 10) Comparaison avec la solution analytique
# =====
model_sk = LinearRegression()
model_sk.fit(X_raw.reshape(-1, 1), y)

print("\n--- Comparaison avec LinearRegression ---")
print(f"LinearRegression : w0 = {model_sk.intercept_:.4f} | w1 = {model_sk.coef_[0]:.4f}")
print(f"SGDRegressor      : w0 = {w0_raw:.4f} | w1 = {w1_raw:.4f}")

```

Cette version utilise SGDRegressor de scikit-learn pour effectuer la descente de gradient automatiquement. Le principe reste le même, mais le calcul du gradient et la mise à jour des paramètres sont gérés par la bibliothèque.

Dans SGDRegressor :

- $\eta = 0.1$ est le taux d'apprentissage,
- α ne veut pas dire la même chose que dans le code manuel,
- ici α correspond à la régularisation.

Donc si on veut une version sans régularisation, il faut mettre : `penalty=None`

9.5. Régularisation : Ridge, Lasso

Ce code compare la régression linéaire classique à deux méthodes de régularisation, Ridge et Lasso, sur un jeu de données contenant à la fois des variables utiles et des variables de bruit. Il montre comment la régularisation permet de limiter l'influence des variables non pertinentes et d'améliorer la robustesse du modèle

```

import numpy as np
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error

# =====

```

```

=== Comparaison des modèles ===

--- Linéaire ---
RMSE = 6,914
Coefficients :
surface      : 104243.7
pieces       : 22472.0
etage        : 7273.3
bruit1       : -895.5

```

```
# 1) Données (avec bruit fort)
# =====
# np.random.seed(42) permet de rendre les expériences
# reproductibles en fixant les valeurs aléatoires.
np.random.seed(42)
m = 80

surface = np.random.randint(30, 150, m)
pieces = np.random.randint(1, 6, m)
etage = np.random.randint(0, 10, m)

# bruit fort (important pour voir l'effet)
bruit1 = np.random.randn(m) * 50
bruit2 = np.random.randn(m) * 50

# prix dépend seulement des vraies variables
y = 20000 + 3000*surface + 15000*pieces + 2000*etage \
    + np.random.randn(m)*10000

X = np.column_stack([surface, pieces, etage, bruit1, bruit2])
noms = ['surface', 'pieces', 'etage', 'bruit1', 'bruit2']

# =====
# 2) Train / Test + Scaling
# =====
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# StandardScaler sert à normaliser les variables pour
# qu'elles aient une moyenne nulle et un écart-type égal à 1.
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# =====
# 3) Modèles (paramètres choisis pour VISUALISER l'effet)
# =====
models = {
    "Linéaire": LinearRegression(),
    "Ridge": Ridge(alpha=5),
    "Lasso": Lasso(alpha=5000)
}

print("\n=== Comparaison des modèles ===")

# =====
# 4) Résultats
# =====
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    rmse = np.sqrt(mean_squared_error(y_test, y_pred))

    print(f"\n--- {name} ---")
    print(f"RMSE = {rmse:,.0f}")

    print("Coefficients :")
    for f, c in zip(noms, model.coef_):
        print(f"{f:10s} : {c:8.1f}")

    print(f"Norme des coefficients = {np.linalg.norm(model.coef_):.1f}")
```

```
bruit2      :   -338.4
Norme des coefficients = 106890.4
```

```
--- Ridge ---
RMSE = 11,855
Coefficients :
surface      :  96508.2
pieces       :  20670.7
etage        :   7005.3
bruit1       : -1601.1
bruit2       :    454.3
Norme des coefficients = 98959.4
```

```
--- Lasso ---
RMSE = 10,862
Coefficients :
surface      :  99479.9
pieces       :  17283.8
etage        :   2629.9
bruit1       :    -0.0
bruit2       :    0.0
Norme des coefficients = 101004.4
```

Les valeurs de α ici sont volontairement élevées pour rendre l'effet visible ; elles ne correspondent pas nécessairement aux meilleures performances prédictives.

En pratique, on choisit α par validation croisée.

Linéaire

- garde toutes les variables
- attribue aussi un poids au bruit,
- peut donc être plus sensible au surapprentissage.

Ridge

- tous les coefficients diminuent
effet "compression"

Lasso

- coefficients de bruit $\rightarrow 0$
sélection automatique

Remarque : sur cet exemple, la régression linéaire donne la meilleure RMSE, mais l'objectif principal est ici d'illustrer l'effet de la régularisation sur les coefficients.

📌 **Fil conducteur** — Immobilier : dans un problème immobilier comportant de nombreuses variables explicatives (surface, étage, orientation, proximité des transports, bruit du quartier, etc.), le Lasso peut aider à identifier automatiquement les variables les plus influentes sur le prix.